Windows passing MinGW passing Ubuntu passing macOS passing

fineFTP Server

FineFTP is a minimal FTP server library for Windows and Unix flavors. The project is CMake based and only depends on asio, which is integrated as git submodule. No boost is required.

You can easily embed this library into your own project in order to create an embedded FTP Server. It was developed and tested on Windows 10 (Visual Studio 2015 and newer, MinGW) and Ubuntu 18.04 - 22.04 (gcc 7.4.0 - 11.2.0). It should also run fine on macOS.

Dependencies

- asio

Features

- FTP Passive mode (the only mode you need nowadays)
- Listing directories
- Uploading and downloading files
- Creating and removing files and directories
- User authentication (and anonymous user without authentication)
- Individual local home path for each user
- Access control on a per-user-basis
- UTF8 support (On Windows MSVC only)

fineFTP does not support any kind of encryption. You should only use fineFTP in trusted networks.

Example

Using fineFTP in your application is simple. Just create an FtpServer object, add one or multiple users and start the server.

```
#include <fineftp/server.h>
#include <thread>
```

```cpp
int main() {
  // Create an FTP Server on port 2121. We use 2121 instead of the default port
  // 21, as your application would need root privileges to open port 21.
  fineftp::FtpServer ftp_server(2121);

  // Add the well known anonymous user. Clients can log in using username
  // "anonymous" or "ftp" with any password. The user will be able to access
  // your C:\ drive and upload, download, create or delete files. On Linux just
  // replace "C:\\" with any valid path. FineFTP is designed to be cross-platform.
  ftp_server.addUserAnonymous("C:\\", fineftp::Permission::All);

  // Start the FTP Server with a thread-pool size of 4.
  ftp_server.start(4);

  // Prevent the application from exiting immediately
  for (;;) std::this_thread::sleep_for(std::chrono::milliseconds(100));
  return 0;
}
```

How to checkout and build

There is an example project provided that will create an FTP Server at C:\ (Windows) or / (Unix).

1. Install cmake and git / git-for-windows
2. Checkout this repo and the asio submodule console `git clone https://github.com/eclipse-ecal/fineftp-server.`
   `cd fineftp-server` `git submodule init` `git submodule update`
3. CMake the project *(Building as debug will add some debug output that is helpful so see if everything is working)* console `mkdir _build` `cd _build` `cmake .. -DCMAKE_BUILD_TYPE=Debug -DCMAKE_INSTALL_PREFIX=_install`
4. Build the project
   - Linux: make
   - Windows: Open _build\fineftp.sln with Visual Studio and build the example project

5. Start fineftp_example / fineftp_example.exe and connect with your favorite FTP Client (e.g. FileZilla) on port 2121 *(This port is used so you don't need root privileges to start the FTP server)*

CMake Options

You can set the following CMake Options to control how fineFTP Server is built:

Option	Type	Default	Explanation
FINEFTP_SERVER_BUILD_SAMPLES	BOOL	ON	Build the fineFTP Server sample project.
FINEFTP_SERVER_BUILD_TESTS	BOOL	OFF	Build the the fineftp-server tests. Requires C++17. For executing the tests, curl must be available from the PATH. For Windows, additionally Powershell and for Linux / macOS the ftp command or python3 with ftplib is used to test the STOU command, that is unsupported by curl.
FINEFTP_SERVER_USE_BUILTIN_ASIO	BOOL	ON	Use the builtin asio submodule. If set to OFF, asio must be available from somewhere else (e.g. system libs).
FINEFTP_SERVER_USE_BUILTIN_GTEST	BOOL	ON *(when building tests)*	Use the builtin GoogleTest submodule. Only needed if FINEFTP_SERVER_BUILD_TESTS is ON. If set to OFF, GoogleTest must be available from somewhere else (e.g. system libs).
BUILD_SHARED_LIBS	BOOL		Not a fineFTP Server option, but use this to control whether you want to have a static or shared library.

How to integrate in your project

Option 1: Integrate as binaries

1. Download the latest release from the releases page or compile the binaries yourself.
2. Add the fineFTP Server directory to your CMAKE_PREFIX_PATH:

```
cmake your_command_line -DCMAKE_PREFIX_PATH=path/to/fineftp/install/dir
```

Option 2: Integrate as source

1. Make the fineFTP Server directory available in your project. You can either add it as a git submodule, or use CMake FetchContent to download it.

2. Add it to your CMake Project:
    - **Either** by adding the top-level CMakeLists.txt to your project

      ```
      add_subdirectory(path/to/fineftp-server)
      ```

      This which will inherit some behavior:

      - You can use the CMake options described below
      - You will get the asio version shipped with fineFTP
      - The debug / minsize / relwithdebinfo postfix will be set automatically

    - **Or** if you want to get a very clean version, which doesn't set any unnecessary options, include the fineftp-server/server subdirectory:

      ```
      add_subdirectory(path/to/fineftp-server/server)
      ```

      You have to provide the required asio target on your own.

Link against fineFTP Server

```
find_package(fineftp REQUIRED)
target_link_libraries(your_target PRIVATE fineftp::server)
```

Contribute

Awesome, you want to contribute to FineFTP? Here is how you can do that!

- Leave us a star ⍓ (That's GitHub money!)
- Create an issue and write about a feature you would like or a bug you have found (maybe we will find some spare time to implement it ⍓)
- Fork this repository, implement the feature yourself and create a pull request